Exploratory Data Analysis (EDA) for IMDB Movie Dataset

Lab 3 | Shreyana Adhikari

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

1: Display a few rows from the dataset

In [2]:
```
df = pd.read_csv("IMDB_Movie_Data.csv")
df.head()
```

Out[2]:

	Rank	Title	Genre	Description	Director	
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	C
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott	L Gree
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James Taylo
3	4	Sing	Animation,Comedy,Family	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	McCo Wit
4	5	Suicide Squad	Action,Adventure,Fantasy	A secret government agency recruits some of th...	David Ayer	W Leto,

In [3]:
```
df.shape
```

Out[3]: (1000, 12)

2: Identify the data type of each column

In [4]:
```
df.dtypes
```

Out[4]:
```
Rank                    int64
Title                  object
Genre                  object
Description            object
Director               object
Actors                 object
Year                    int64
Runtime (Minutes)       int64
Rating                float64
Votes                   int64
Revenue (Millions)    float64
Metascore             float64
dtype: object
```

3: Check for missing values

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   Rank                1000 non-null   int64  
 1   Title               1000 non-null   object 
 2   Genre               1000 non-null   object 
 3   Description         1000 non-null   object 
 4   Director            1000 non-null   object 
 5   Actors              1000 non-null   object 
 6   Year                1000 non-null   int64  
 7   Runtime (Minutes)   1000 non-null   int64  
 8   Rating              1000 non-null   float64
 9   Votes               1000 non-null   int64  
 10  Revenue (Millions)  872 non-null    float64
 11  Metascore           936 non-null    float64
dtypes: float64(3), int64(4), object(5)
memory usage: 93.9+ KB
```

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
Rank                    0
Title                   0
Genre                   0
Description             0
Director                0
Actors                  0
Year                    0
Runtime (Minutes)       0
Rating                  0
Votes                   0
Revenue (Millions)    128
Metascore              64
dtype: int64
```

4: Check for duplicate values and delete if any

In [7]:
```
df.duplicated().sum()
```

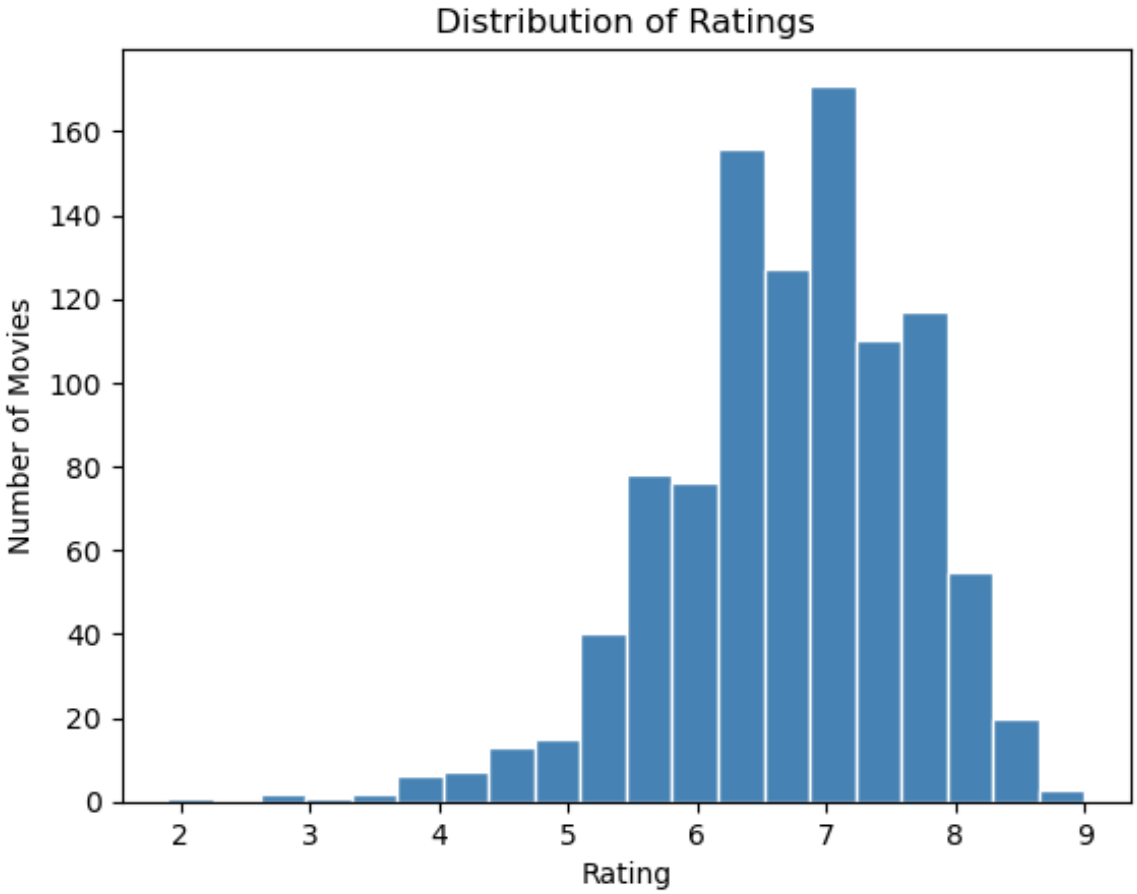
Out[7]:
```
np.int64(0)
```

In [8]:
```
df = df.drop_duplicates()
df.shape
```

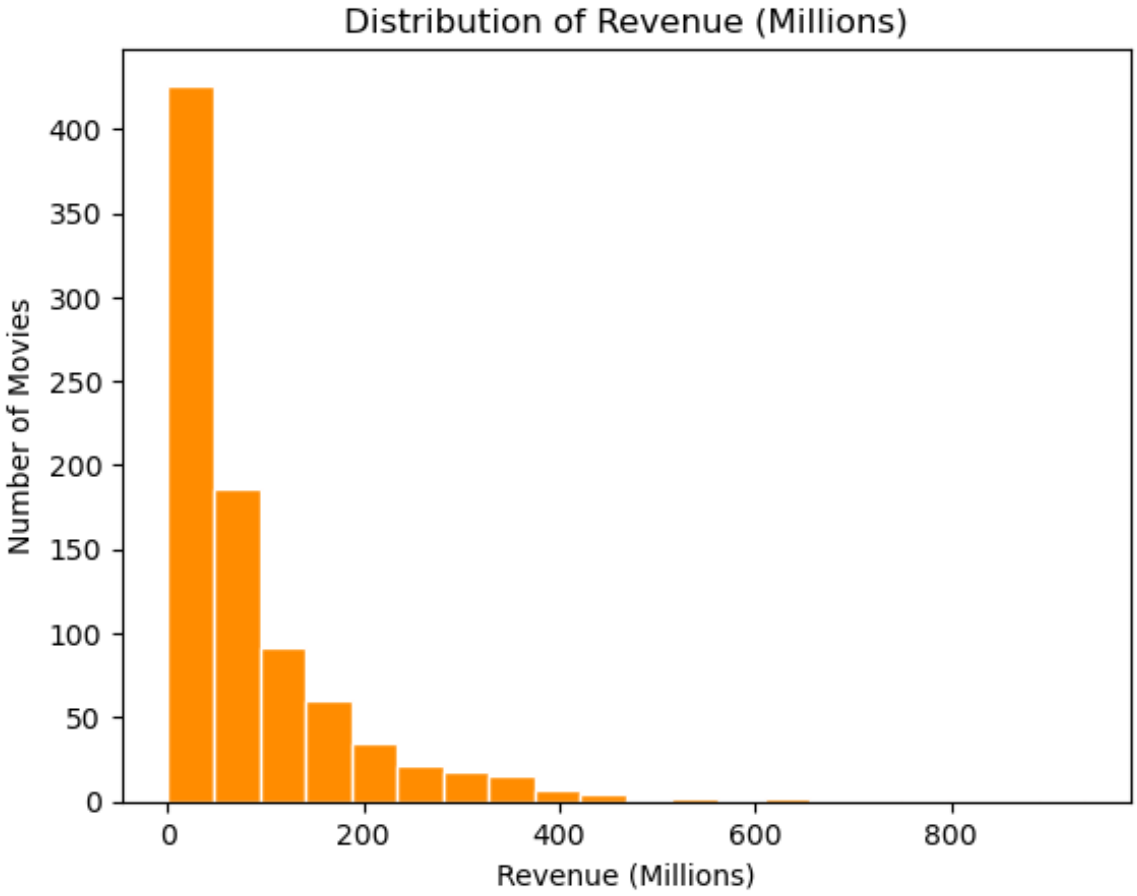
Out[8]:
```
(1000, 12)
```

5: Create histograms or bar charts for numerical and categorical columns to visualize their distributions

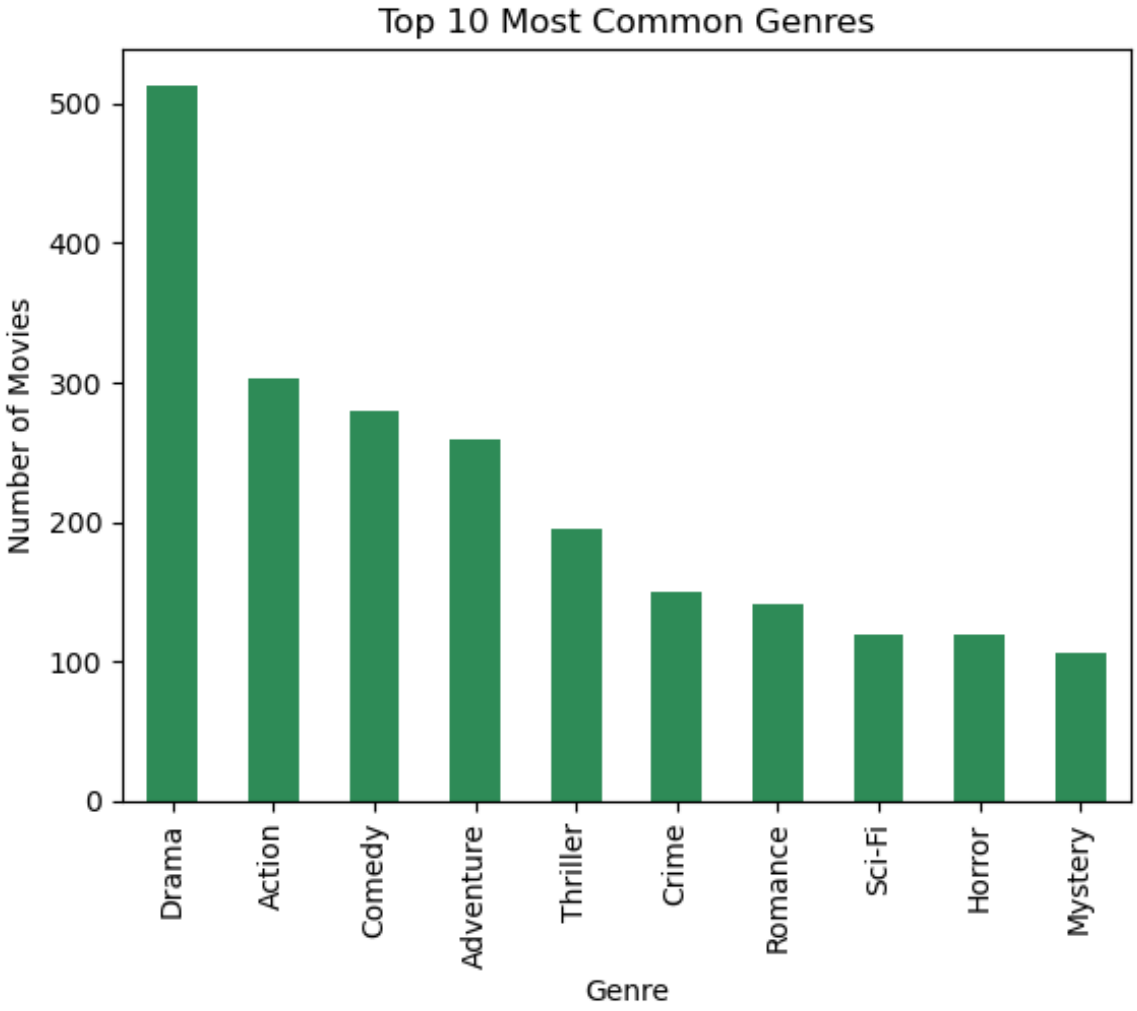
In [9]:
```
plt.hist(df["Rating"], bins=20, color="steelblue", edgecolor="white
plt.title("Distribution of Ratings")
plt.xlabel("Rating")
plt.ylabel("Number of Movies")
plt.show()
```



In [10]:
```
plt.hist(df["Revenue (Millions)"].dropna(), bins=20, color="darkora
plt.title("Distribution of Revenue (Millions)")
plt.xlabel("Revenue (Millions)")
plt.ylabel("Number of Movies")
plt.show()
```

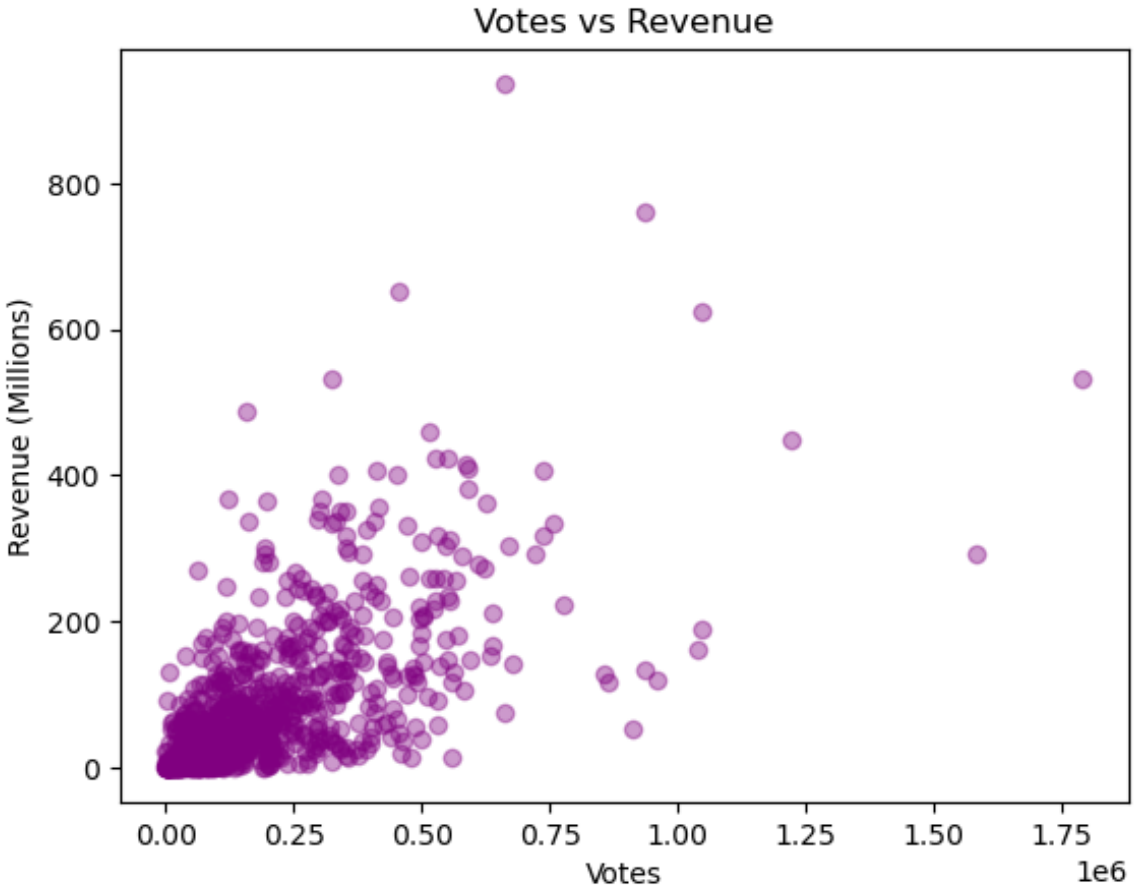


In [11]:
```
all_genres = df["Genre"].str.split(",").explode().str.strip()
all_genres.value_counts().head(10).plot(kind="bar", color="seagreen
plt.title("Top 10 Most Common Genres")
plt.xlabel("Genre")
plt.ylabel("Number of Movies")
plt.show()
```

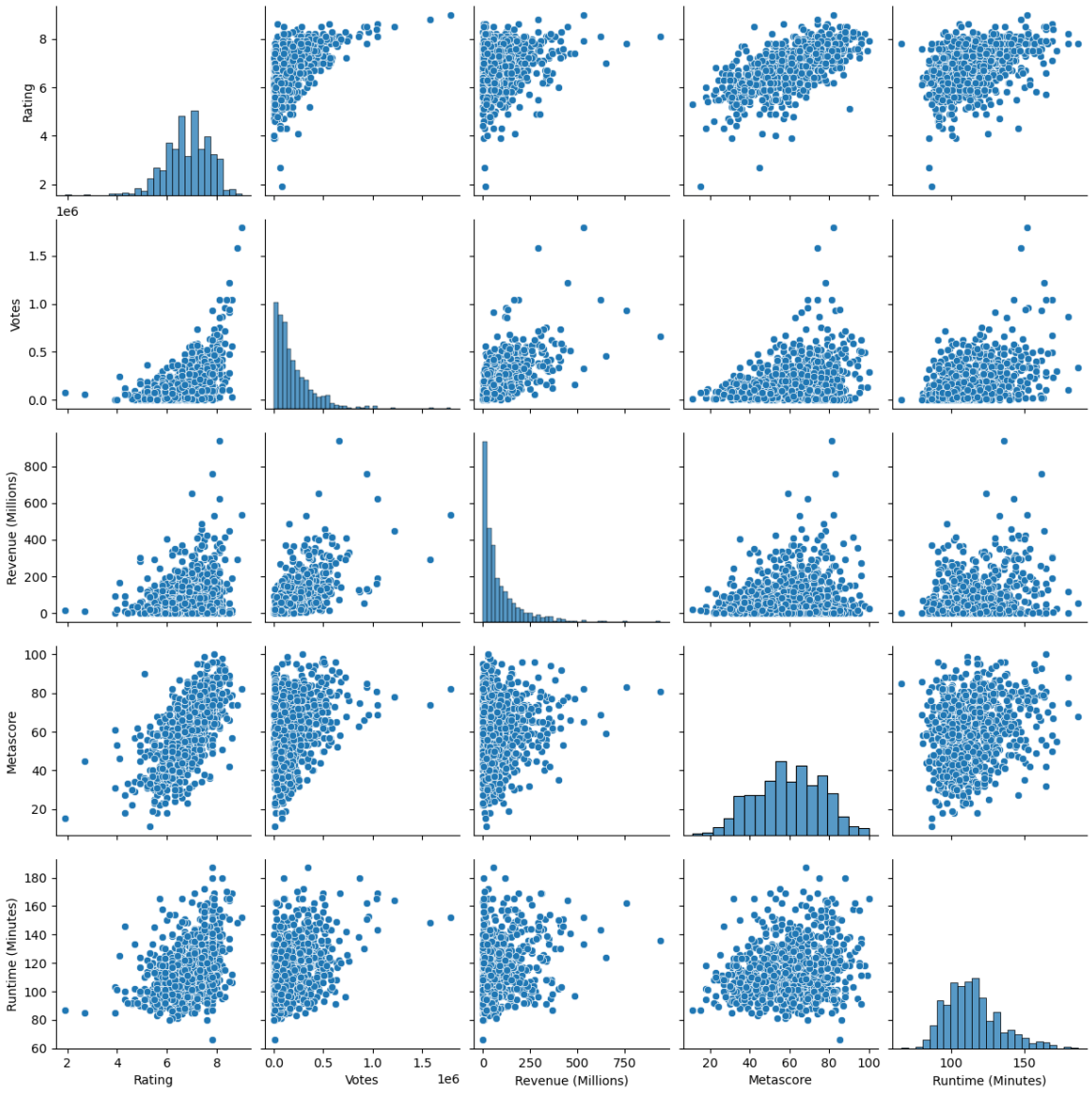


6: Generate scatter plots or pair plots to explore relationships between pairs of numerical variables

In [12]:
```
plt.scatter(df["Votes"], df["Revenue (Millions)"], alpha=0.4, color
plt.xlabel("Votes")
plt.ylabel("Revenue (Millions)")
plt.title("Votes vs Revenue")
plt.show()
```



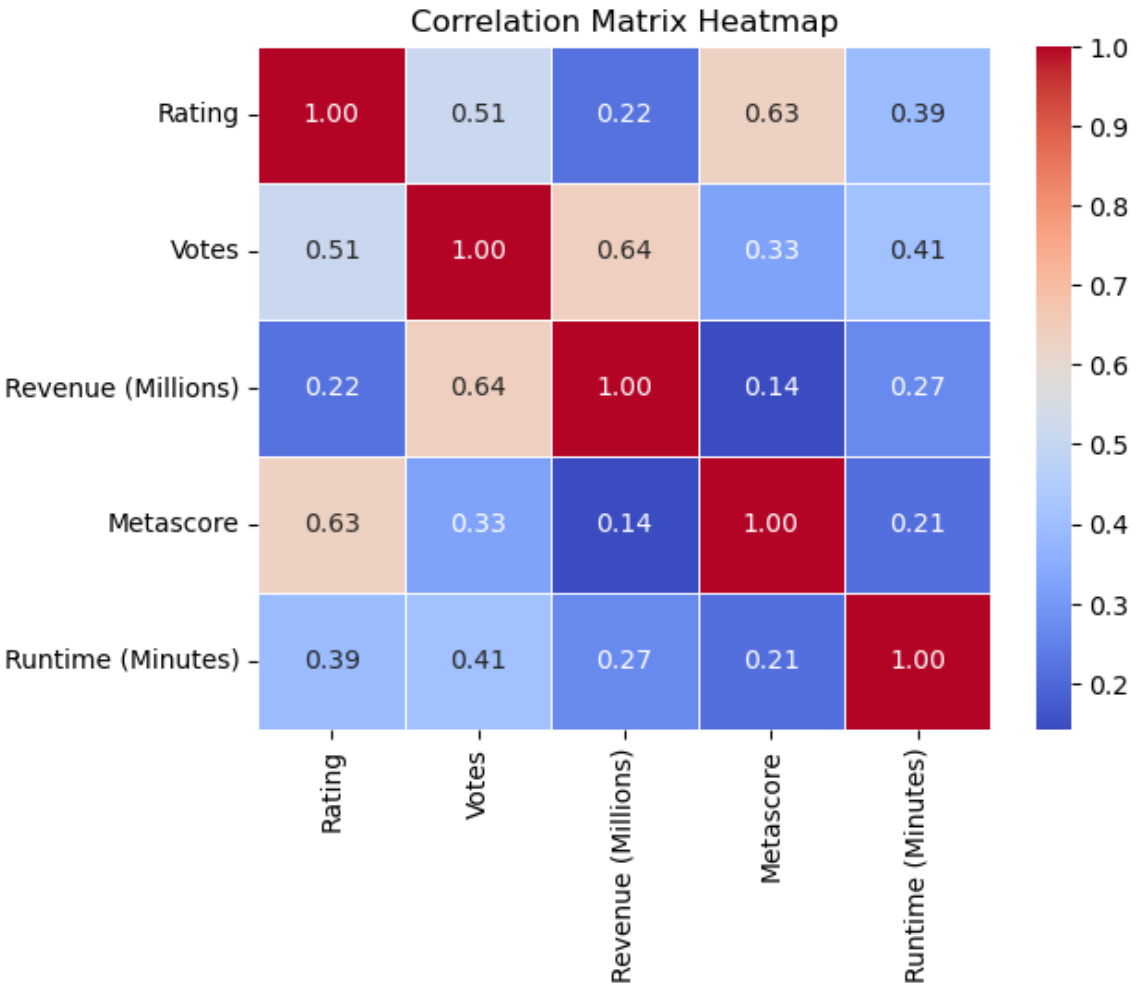
In [13]:
```
sns.pairplot(df[["Rating", "Votes", "Revenue (Millions)", "Metascor
plt.show()
```



7: Create a heatmap of the correlation matrix

In [14]:
```
num_cols = ["Rating", "Votes", "Revenue (Millions)", "Metascore", "
corr_matrix = df[num_cols].corr()

sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", li
plt.title("Correlation Matrix Heatmap")
plt.show()
```


Correlation Matrix Heatmap
Rating
Votes
Revenue (Millions)
Metascore
Runtime (Minutes)
1.00 0.51 0.22 0.63 0.39
0.51 1.00 0.64 0.33 0.41
0.22 0.64 1.00 0.14 0.27
0.63 0.33 0.14 1.00 0.21
0.39 0.41 0.27 0.21 1.00
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2